Summary of "Serving Systems for Agents"

Ruiqi Chen (ruiqich), Michela Marchini (mchini), Marilyn Rego (mrego), Ellie Seehorn (seehorn)

Research Direction of Field:

Research Problem and its Significance: The research problem concerns efficient execution of LLM applications whose behavior extends beyond individual text-generation requests. Applications like agentic systems involve branching, repeated model invocations, external tools, and persistent intermediate state. Supporting such applications require coordination between application logic, inference execution and infrastructure management. There are two major limitations that arise at the different levels of the serving stack. At the inference level, conventional interfaces provide limited application-specific control over generation, KV-cache management, and I/O. At the workflow level, model choices and resource allocations are often configured separately from application requirements and runtime conditions. These limitations can introduce redundant computation, increase latency, reduce resource utilization while making service-level objectives (SLOs) difficult to achieve.

State of Prior Related Work: Earlier serving systems primarily addressed request-level efficiency. Orca introduced iteration-level scheduling, and vLLM improved KV-cache memory management through PagedAttention. These systems improved the execution of individual generation requests and established a foundation for subsequent application-aware optimizations. Other work examined application expressiveness and structure. For example, SGLang exposes relationships among generation requests, enabling optimizations such as KV-cache reuse and application-aware scheduling. These approaches provide increased visibility into the application behavior, although their interfaces did not offer the scope of inference control proposed by Pie.

Workflow frameworks such as LangChain and DSPy support the composition and optimization of multi-step applications. However, workflow development and infrastructure management largely remain separate. Related systems addressed model selection, scheduling, or resource adaptation, without jointly coordinating the full set of workflow and infrastructure decisions considered by Murakkab. The field is therefore developed through overlapping directions, with the scope of optimization expanding from individual requests to structure applications and their resource requirements.

Contributions of the papers: Pie addresses control within the inference-serving system. It decomposes generation into API-accessible operations and executes application-defined WebAssembly programs, termed inferlets, that coordinate generation, KV-cache management, and I/O. Its contribution is an execution abstraction that permits application-specific inference policies without requiring modifications to the serving engine. Relative to prior approaches, Pie extends programmability from generation behavior to the management of inference operations and state. Murakkab addresses configuration and resource management across agentic workflows. It separates workflow specification from execution configuration and uses profile-guided optimization and runtime adaptation to coordinate workflow settings, model selection, and hardware allocation under SLO constraints. Its contribution is to make workflow structure and requirements available to infrastructure-level optimization. The papers address complementary levels of the serving stack. Pie provides application control over inference execution, while Murakkab coordinates execution configurations and resources across workflows. Together, they examine how application information can inform decisions previously confined to separate system layers.

Subsequent work built on lecture papers: Aragog examines limitations of configuration-selection approaches, including Murakkab, and introduces stage-level model assignment based on current serving conditions. It extends this line of research toward adaptation during individual workflow executions, accounting for changes in system load after a request begins.

Lecture Papers: Pie (SOSP 2025)

Problem: Existing serving engines couple caching and decoding policies to a fixed generation loop. Applications cannot directly control which reasoning branches retain their KV cache or how generation interacts with tools. Client coordination adds round trips and can repeat prefill.

Solution: Pie delegates generation to inferlets, user programs running in WebAssembly. APIs expose embedding, forward passes, sampling, and KV pages, allowing custom decoding and cache reuse across tool

calls. Command queues expose dependencies so the scheduler can batch compatible calls of the same type within and across queues. Specialized policies require application knowledge and extra programming, although support libraries simplify common operations.
Evaluation: Tests use Llama models from 1B to 8B on one 24 GB L4 against vLLM and SGLang, all using FlashInfer. An agent combining documentation caching, context removal, and overlapping tool calls reaches 3.5 times vLLM's throughput. Plain completion adds 2.4% to 11.4% latency per token, mainly from lost GPU pipelining
Shortcomings: The single GPU evaluation leaves distributed scheduling and KV placement untested. Resource contention is handled by terminating newer inferlets, with quotas and stronger security controls left for future work.
Murakkab (OSDI 2026)
Problem: Workflow settings, model selection, and hardware allocation are configured separately even though they jointly determine quality and resource use. Scaling a fixed deployment cannot adjust choices such as frame count or debate rounds when request requirements change.
Solution: Murakkab separates workflow logic from execution settings. An LLM orchestrator maps tasks to an executor library and forms a DAG. Workflow profiles record accuracy and token load, while model profiles measure performance, energy, and cost across hardware and parallelism settings. A mixed-integer linear program jointly chooses workflow settings, models, hardware allocations, and shared instances under service-level objectives (SLOs). Periodic reoptimization and faster auto-scaling adapt to demand.
Evaluation: Experiments use vLLM on Azure A100 and H100 VMs. Trace tests map 24 hours of LLM arrivals to video Q/A and code generation. Against LangGraph with the same auto-scaler, joint optimization and sharing reduce GPU requirements from 2,472 to 912 and cost from $112.3k to $47.2k while meeting selected SLOs. Minimizing energy can instead increase cost.
Shortcomings: Reliable configuration choices depend on representative profiles, but profile transfer is tested only within math. Dynamic coding is evaluated for accuracy and token use, not jointly with hardware allocation and auto-scaling
Lecture Discussion: Discussion of PIE focused mainly on the tradeoffs the paper introduces.

1. Fairness — Pie uses FCFS when KVPages run out which poses a "fairness tax" under resource contention, though nothing gets starved. Note: when there is free space nobody is affected.
2. Security — PIE encourages user control, so APIs are exposed for programmability. An analogy is C where developers get control of memory allocation, but then must remember to call free.
3. Overhead — Extra checks for correctness and leaks would add overhead that could undermine advantages. There are not many of these checks, but, this is a throughput paper and security and latency are not priorities of the authors. Think of moving from C to C++: to protect from raw pointers you invent smart pointers, but smart pointers carry a little overhead.
4. User control — With Pie, users are calling functions step-by-step and the server loses control and sight of the full request. This can raise concerns for optimization, more complex KV caching in specialized settings, such as with MOE models, and errors due to poor user code. Optimization and full vLLM functionality are possible, but the onus is on the user. Notably, though, the audience here isn't typical end users, it's people who know the application well.

Discussion of Murakkab focused on its fixed and generalizable components and its limitations.

1. Fixed — Murakkab follows workflows developers define. It is not designed for prediction, but rather is ideal for large companies with many fixed workflows that need to be deployed.
2. Generalizable — Murakkab can be used for many applications and the SLOs are per-workflow.
3. Limitations — Murakkab only supports sequential workflows, but workflows may not always be strict and deterministic in this way. The system struggles with increased variance and non-determinism, so it is less ideal in applications with many branches. Additionally, they note LLM behavior is out of their control, and they did not evaluate rates of LLM mistakes and/or any debugging in such scenarios because the system is not intended for public use.

Proposal for Follow-Up Work: Pie's fine-grained serving control can require low-level programming expertise to maintain throughput and latency. Relatedly, Murakkab automatically optimizes developer-defined workflows, but domain-specific requirements may be difficult to express through its configuration choices. Both may therefore challenge developers with strong domain knowledge but limited serving expertise, raising the question: **how can agent serving systems leverage programmability to enable developers with strong domain knowledge but limited serving expertise to express their requirements while preserving resource optimization under those requirements?**

Techniques from end-user programming may fill this gap, specifically mixed-initiative constraint specification (drawing on SMARTedit) and programming by example with counterexamples (as seen in FlashRelate). Programming by example with counterexamples is relevant because domain experts could likely identify appropriate and inappropriate behavior more easily than they could specify an ideal policy to realize it. Further, mixed- initiative constraint specification could allow a user to naturally express what their system priorities are, while keeping the line between preference and hard constraint clear. Developers would describe their workflows, priorities, and hard constraints in natural language, with targeted questions informing an initial policy explained in application-specific terms and open to inspection and revision. They would inspect outputs and execution traces from representative requests and label acceptable and unacceptable behavior—for example, rejecting plausible code returned without verification, prompting clarification of whether verification is always mandatory or only required in particular circumstances. Confirmed requirements would restrict eligible configurations, preferences would rank them, and profiling would guide resource-efficient selection while identifying conflicts with the stated constraints.

Evaluation proceeds in two parts: a developer-oriented evaluation and a system benchmark. In the developer oriented evaluation, we would work with several agent developers (ideally n=20-30) who have expertise in their agent's domain but not in serving systems. They would author policies using our system, Murakkab and Pie's supported abstractions: comparing requirement fidelity (with requirements pre-documented by developers), authoring time, and developer-rated ease of use. The resulting policies would then be benchmarked on held-out requests and changing workloads, measuring cost, throughput, tail latency, task quality, constraint violations, and profiling/adaptation overhead. Reference policies authored by a separate set of participants with expertise in agent serving would help distinguish inefficiencies arising from authoring difficulty from limitations of the serving mechanism.

Reflection on AI Use:

Marilyn: I worked on section 1 of the summary. I used GPT-6.1 Sol model to review my draft and revise it, focusing on the research problems, related work and identifying the gaps in my draft. It was useful in finding correlations between the works but it also introduced some information beyond what I gave in my draft. So I had to review the information, check the papers, and make sure that the information is relevant and improves my draft.

Ruiqi: Opus 5.5 helped organize my notes, but shortened some points too far (Section 6, Opus draft). It reduced Pie's tradeoff to manual resource management and made Murakkab's 60-minute interval sound fixed. I had to go back to my notes of these two readings and tried to have a balance trade-off between providing more information that Opus 5.5 not provide and the pages limitations. I think Opus 5.5 tried to follow the guidance of the pages limitation in my prompt so it do more on pages limitations instead of following the Section 2 requirements.

Ellie: I worked on section 4. I first wrote up my idea independently, and then I passed my writeup, the class slides, and both papers to ChatGPT and asked it to poke holes in the follow-up, which I used to refine it and strengthen it as a research idea. I found it to be helpful with identifying where my idea missed points about the papers (especially some aspects of Murakkab that I had become confused about throughout the Q&A session), but less useful for actually grounding the changes in the rest of this summary because it had access only to the papers, not our class discussion. For instance, the part about making Pie making it the end-user's responsibility to get allocation decisions right was a big part of our discussion, but ChatGPT rejected I could claim that because it wasn't there for our discussion. I also used it to suggest areas to cut, pretty successfully.